

PROJECT DOCUMENTATION

# AI-Powered Brain Tumor Detection, Classification, and Segmentation System Using Deep Learning

Python • TensorFlow • FastAPI • React.js • OpenCV • U-Net

**Developer:** Aryan Sengar (Roll No. 23CSU055)

**Mentor:** Dr. Ashima Rani, Dept. of Computer Science & Engineering

**Project Type:** Third Year B.Tech CSE Project

**Domain:** Medical Imaging • Deep Learning • Full-Stack Web Application

**Academic Year:** 2025–26 | May 2026

## Abstract

This project implements an end-to-end AI-powered web system for automated brain tumor analysis from MRI scans. Three deep learning models are integrated into a sequential diagnostic pipeline: a custom CNN for binary tumor detection (97.3% test accuracy), a MobileNetV2 transfer learning model for four-class tumor type classification (90.0% accuracy), and a U-Net encoder-decoder for pixel-wise tumor segmentation (Dice score 0.80+). The system is deployed through a React.js frontend and a FastAPI backend, providing single-image analysis, batch processing, automated PDF report generation, QR code sharing, and analysis history management — all operational in a CPU-only environment.

|                                                |                                                     |                                                     |                                            |                                           |
|------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------|--------------------------------------------|-------------------------------------------|
| <div>97.3%</div> <div>Detection Accuracy</div> | <div>90.0%</div> <div>Classification Accuracy</div> | <div>0.80+</div> <div>Segmentation Dice Score</div> | <div>~23K</div> <div>Training Images</div> | <div>3-Stage</div> <div>AI Pipeline</div> |
|------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------|--------------------------------------------|-------------------------------------------|

# 1 • Introduction

Brain tumors affect millions worldwide and require fast, accurate diagnosis for effective treatment. Manual MRI interpretation by specialist radiologists is time-consuming, prone to inter-observer inconsistency, and limited by specialist availability — particularly in resource-constrained settings.

This project delivers a complete AI diagnostic system that automates the three core analysis tasks: detecting whether a tumor is present, identifying its type, and delineating its boundaries at pixel level. The system packages these capabilities into a professional web application accessible to any clinical user with a browser, without requiring specialist technical knowledge.

## Pipeline Overview

| Stage | Task                | Model                                       | Result                |
|-------|---------------------|---------------------------------------------|-----------------------|
| 1     | Binary Detection    | Custom CNN (Grayscale 128×128)              | <b>97.3% Accuracy</b> |
| 2     | Type Classification | MobileNetV2 Transfer Learning (RGB 224×224) | <b>90.0% Accuracy</b> |
| 3     | Tumor Segmentation  | U-Net Encoder-Decoder (RGB 128×128)         | <b>Dice 0.80+</b>     |

Table 1.1 – Three-stage diagnostic pipeline summary

## 2 • Problem Statement

Current brain tumor diagnosis relies entirely on manual MRI review by expert neuroradiologists. This creates four compounding problems in clinical practice:

- Diagnostic delay — Expert manual review of hundreds of MRI slices can take hours per patient, delaying treatment initiation for time-critical conditions.
- Inter-observer variability — Subtle lesions are interpreted differently between radiologists, producing inconsistent diagnoses for similar presentations.
- Specialist scarcity — Neuroradiologists are globally scarce, particularly in rural and developing-world healthcare settings where patients may have no access to expert review.
- Manual segmentation burden — Precise tumor boundary delineation for surgical planning is highly skilled, slow, and infeasible at scale without automation.

This system addresses all four problems by automating detection, classification, and segmentation through a unified, web-accessible AI pipeline that delivers results in seconds.

## 3 • Objectives

1. Build a CNN binary classifier to detect tumor vs. healthy brain MRI scans with clinically meaningful accuracy.
2. Build a MobileNetV2 multi-class classifier to identify Glioma, Meningioma, Pituitary, and No Tumor subtypes.
3. Build a U-Net segmentation model to produce pixel-level tumor boundary masks, evaluated by Dice coefficient.
4. Integrate all three models into a sequential end-to-end pipeline with a single unified API call.
5. Implement a FastAPI backend exposing five RESTful prediction and reporting endpoints.
6. Build a React.js frontend providing single-image analysis, batch processing, PDF reports, QR sharing, and history management.
7. Validate all models using standard metrics and benchmark against baseline methods.

## 4 • Technology Stack

| Layer             | Technology                                  | Role                                         |
|-------------------|---------------------------------------------|----------------------------------------------|
| Frontend          | React.js, Vite, Tailwind CSS, Framer Motion | Responsive single-page web application       |
| HTTP Client       | Axios, React Router DOM, Recharts           | API communication, routing, dashboard charts |
| Backend           | Python, FastAPI, Uvicorn                    | RESTful API server, model serving            |
| ML Framework      | TensorFlow 2.x / Keras                      | Model training and inference                 |
| Image Processing  | OpenCV, Pillow, NumPy                       | Preprocessing pipeline                       |
| Report Generation | ReportLab                                   | Automated PDF clinical reports               |
| Data Science      | pandas, scikit-learn, matplotlib, seaborn   | Metrics, evaluation, visualisation           |
| Storage           | Browser localStorage                        | Client-side analysis history (no server DB)  |
| Dev Tools         | VS Code, Node.js, npm, GitHub               | Development and version control              |

Table 4.1 – Complete technology stack

## 5 • System Architecture

The system follows a three-tier architecture with clean separation between the user interface, API logic, and model inference layers.

| Tier    | Components and Responsibilities                                                                                                                                                                                       |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Layer 1 | Frontend (React.js) — DropZone upload, ResultPanel, Batch Analysis, Dashboard, History Page, QR Code module. Communicates with Layer 2 via Axios HTTP/JSON.                                                           |
| Layer 2 | API (FastAPI + Uvicorn) — Image validation, preprocessing, model invocation, post-processing, PDF generation. Five endpoints: /predict/binary, /predict/type, /predict/segmentation, /predict/full, /generate-report. |
| Layer 3 | Model Layer (TensorFlow) — Three pre-loaded models: CNN Binary Classifier (128×128 grayscale), MobileNetV2 Classifier (224×224 RGB), U-Net Segmenter (128×128 RGB). Loaded at server startup, no cold-start delay.    |

Table 5.1 – Three-tier architecture overview

### End-to-End Request Flow

8. User uploads MRI image via the React DropZone component.
9. Axios sends a POST request with image as FormData to the FastAPI /predict/full endpoint.
10. FastAPI preprocesses the image (resize, normalise, colour-space convert) for each model's input specification.
11. CNN binary detector runs first — returns Tumor or Healthy with confidence score.
12. If Tumor is detected: MobileNetV2 runs — returns four-class probability distribution.
13. U-Net runs — returns a pixel-wise binary mask and tumor area percentage.
14. Results compiled into JSON response and rendered in the React ResultPanel.
15. On demand: /generate-report triggers ReportLab PDF generation with embedded results and QR code.

## 6 • Dataset & Preprocessing

### Datasets

| Dataset             | Format       | Classes                                    | Notes                                                |
|---------------------|--------------|--------------------------------------------|------------------------------------------------------|
| Binary Detection    | PNG / JPG 2D | Tumor / Healthy                            | ~22,946 total images; 1,924 test samples             |
| Type Classification | PNG (RGB)    | Glioma / Meningioma / Pituitary / No Tumor | Kaggle — Masoud Nickparvar dataset                   |
| Segmentation        | PNG          | Tumor mask / Background                    | Kaggle — Masoud <a href="#">Nikhil Tomar</a> dataset |

Table 6.1 – Dataset summary

### Preprocessing Pipeline

| Step               | CNN (Binary)              | MobileNetV2               | U-Net                          |
|--------------------|---------------------------|---------------------------|--------------------------------|
| Colour Space       | Grayscale                 | RGB (retained)            | RGB (3-channel)                |
| Input Size         | $128 \times 128 \times 1$ | $224 \times 224 \times 3$ | $128 \times 128 \times 3$      |
| Normalisation      | Divide by 255             | Divide by 255             | Divide by 255                  |
| Augmentation       | Rotate, Flip, Zoom        | Rotate, Flip, Zoom, Shear | Paired image-mask augmentation |
| Imbalance Handling | Balanced sampling         | Class-weighted loss       | Threshold masks at 0.5         |

Table 6.2 – Preprocessing configuration per model



## 7 • Binary Classification Model (CNN)

### Architecture

- Input:  $128 \times 128 \times 1$  grayscale MRI
- Conv2D block 1: 32 filters,  $3 \times 3$ , ReLU  $\rightarrow$  MaxPool  $2 \times 2$
- Conv2D block 2: 64 filters,  $3 \times 3$ , ReLU  $\rightarrow$  MaxPool  $2 \times 2$
- Flatten  $\rightarrow$  Dense (128 units, ReLU)  $\rightarrow$  Dropout (0.5)
- Output: Dense (2 units, Sigmoid) — binary class probabilities

### Training Configuration

| Parameter     | Value                                                       |
|---------------|-------------------------------------------------------------|
| Optimiser     | Adam                                                        |
| Loss Function | Binary Cross-Entropy                                        |
| Epochs        | 10                                                          |
| Batch Size    | 433 (CPU-memory optimised)                                  |
| Callbacks     | EarlyStopping (patience=5) • ReduceLROnPlateau (factor=0.5) |

Table 7.1 – CNN binary classifier training configuration

### Results

| Metric                        | Value        |
|-------------------------------|--------------|
| Test Accuracy (1,924 samples) | <b>97.3%</b> |
| Training Accuracy (Epoch 10)  | 99.6%        |
| Precision (Tumor class)       | 97%          |
| Recall (Tumor class)          | 97%          |
| F1-Score (Tumor class)        | <b>0.97</b>  |

Table 7.2 – CNN binary classifier – test set performance

## 8 • Multi-Class Classification Model (MobileNetV2)

### Architecture

- Base: MobileNetV2 pretrained on ImageNet — all base layers frozen (transfer learning).
- Input:  $224 \times 224 \times 3$  RGB
- GlobalAveragePooling2D → Dense (128 units, ReLU) → Dropout (0.3)
- Output: Dense (4 units, Softmax) — class probability distribution
- Classes: Glioma • Meningioma • Pituitary • No Tumor

### Training Configuration

| Parameter       | Value                                                       |
|-----------------|-------------------------------------------------------------|
| Optimiser       | Adam                                                        |
| Loss Function   | Categorical Cross-Entropy                                   |
| Epochs          | 15                                                          |
| Batch Size      | 175 (CPU-memory optimised)                                  |
| Augmentation    | Rotation, Zoom, Horizontal Flip, Shear                      |
| Class Imbalance | Class-weighted loss • Balanced augmentation                 |
| Callbacks       | EarlyStopping (patience=5) • ReduceLROnPlateau (factor=0.5) |

Table 8.1 – MobileNetV2 training configuration

### Per-Class Results

| Class                | Precision   | Recall      | F1-Score    | Note                            |
|----------------------|-------------|-------------|-------------|---------------------------------|
| Glioma               | 0.94        | 0.76        | 0.84        | Lowest recall — class imbalance |
| Meningioma           | 0.85        | 0.86        | 0.86        | Balanced performance            |
| Pituitary            | 0.88        | 1.00        | 0.94        | Perfect recall                  |
| No Tumor             | 0.94        | 0.98        | 0.96        | Strong performance              |
| <b>Macro Average</b> | <b>0.90</b> | <b>0.90</b> | <b>0.90</b> | Overall                         |

Table 8.2 – MobileNetV2 per-class classification report (90.0% overall accuracy)

## 9 • Segmentation Model (U-Net)

### Architecture

The U-Net encoder-decoder architecture is used for pixel-wise tumor boundary delineation. Skip connections between encoder and decoder layers allow the model to recover fine spatial detail lost during downsampling — critical for precise tumor boundary prediction.

- Encoder: Repeated Conv2D + Batch Normalisation + ReLU blocks, separated by MaxPooling2D downsampling layers.
- Bottleneck: Deepest feature representation (most abstract context).
- Decoder: Upsampling layers concatenated with corresponding encoder feature maps via skip connections.
- Output: 1×1 Conv2D with Sigmoid activation — binary pixel-wise probability map (128×128).
- Regularisation: Batch Normalisation + Dropout applied across all encoder and decoder blocks.

### Loss Function

A combined Dice + Binary Cross-Entropy (BCE) loss is used. BCE provides stable pixel-level gradient feedback; Dice loss directly optimises the overlap metric used in evaluation.

$$\text{Dice} = 2 \times |A \cap B| / (|A| + |B|)$$

### Training Configuration & Results

| Parameter     | Value                             | Result Metric            |
|---------------|-----------------------------------|--------------------------|
| Optimiser     | Adam                              | Training Dice (Epoch 30) |
| Loss          | Dice + Binary CE                  | <b>0.9164</b>            |
| Epochs        | 30                                | Validation Dice          |
| Batch Size    | 307 (CPU optimised)               | <b>0.8041</b>            |
| Callbacks     | EarlyStopping + ReduceLROnPlateau | Final Dice Score         |
| Reg. Strategy | BatchNorm + Dropout               | <b>0.80+</b>             |

Table 9.1 – U-Net configuration and results

## 10 • Frontend & Backend Implementation

### React.js Frontend

The frontend is a single-page application built with React.js and Vite, styled with Tailwind CSS, animated with Framer Motion, and routed with React Router DOM. Key components:

| Component      | Function                                                                                                |
|----------------|---------------------------------------------------------------------------------------------------------|
| DropZone       | Drag-and-drop MRI upload for single or multiple scans; serialises image as FormData via Axios.          |
| ResultPanel    | Renders detection confidence, tumor type probabilities, segmentation mask overlay, and area percentage. |
| Batch Analysis | Sequential multi-file upload and processing; displays consolidated results per scan.                    |
| Dashboard      | Recharts-powered analytics showing prediction distributions and accuracy trends from history data.      |
| History Page   | Reads and renders full analysis log stored in browser localStorage; no server-side database required.   |
| QR Code Module | Generates a scannable QR code to access system in mobile                                                |

Table 10.1 – React frontend components

### FastAPI Backend

The FastAPI backend (served by Uvicorn) handles all model inference, image preprocessing, and PDF generation. All three models are loaded into memory at server startup to eliminate per-request cold-start delays.

#### Image Preprocessing Flow (per request)

16. Load uploaded image bytes → convert to NumPy array via Pillow.
17. Apply colour-space conversion: grayscale for CNN; retain RGB for MobileNetV2 and U-Net.
18. Resize to model input dimensions using OpenCV bilinear interpolation.
19. Normalise pixel values to [0.0, 1.0] (divide by 255).
20. Expand dimensions to add batch axis: shape (1, H, W, C).

#### PDF Report Content (ReportLab)

- Scan metadata and analysis timestamp
- Binary detection result with confidence score
- Tumor type with class-wise probability table
- Segmentation mask overlay image
- Tumor area as percentage of total scan area

## 11 • API Workflow

| Endpoint                              | Method | Description                                                                                       |
|---------------------------------------|--------|---------------------------------------------------------------------------------------------------|
| <a href="#">/predict/binary</a>       | POST   | Runs CNN binary detection; returns Tumor/Healthy with confidence score.                           |
| <a href="#">/predict/type</a>         | POST   | Runs MobileNetV2; returns four-class probability distribution.                                    |
| <a href="#">/predict/segmentation</a> | POST   | Runs U-Net; returns binary mask and tumor area percentage.                                        |
| <a href="#">/predict/full</a>         | POST   | Executes full three-stage pipeline in a single request; returns all outputs combined.             |
| <a href="#">/generate-report</a>      | POST   | Generates downloadable PDF report with embedded results, mask overlay, and QR code via ReportLab. |

Table 11.1 – FastAPI endpoint reference

All endpoints accept multipart/form-data image uploads and return structured JSON responses. The `/predict/full` endpoint uses conditional branching: if the CNN binary detector returns Healthy (probability < 0.5), the pipeline terminates and returns immediately without invoking the MobileNetV2 or U-Net models, improving response time for healthy scans.

## 12 • Results & Evaluation Metrics

|                            |                      |                                  |                        |                     |
|----------------------------|----------------------|----------------------------------|------------------------|---------------------|
| 97.3%<br>CNN Test Accuracy | 0.97<br>CNN F1-Score | 90.0%<br>MobileNetV2<br>Accuracy | 0.90<br>Macro F1-Score | 0.80+<br>U-Net Dice |
|----------------------------|----------------------|----------------------------------|------------------------|---------------------|

### Comparative Analysis

| Method                         | Task             | Accuracy / Dice | Notes                                             |
|--------------------------------|------------------|-----------------|---------------------------------------------------|
| Baseline CNN                   | Binary Detection | ~85%            | Simple architecture, limited generalisation       |
| ResNet50 Transfer (Literature) | Binary Detection | ~91%            | Strong but slower, GPU-dependent                  |
| VGG16 Transfer (Literature)    | Binary Detection | ~88%            | Moderately competitive                            |
| Our CNN (This Project)         | Binary Detection | 97.3%           | +12.3pp over baseline; CPU-only training          |
| Vanilla U-Net (Baseline)       | Segmentation     | ~0.70 Dice      | No regularisation                                 |
| Our U-Net (This Project)       | Segmentation     | 0.80+ Dice      | +0.10 Dice over baseline; BN + Dropout + Dice+BCE |

Table 12.1 – Comparative model performance summary

## 13 • Key Features

| Feature                      | Implementation Detail                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------|
| <b>3-Stage AI Pipeline</b>   | Detection → Classification → Segmentation in a single /predict/full API call.                |
| <b>Batch Processing</b>      | Multi-file upload interface for sequential analysis of multiple MRI scans in one session.    |
| <b>PDF Report Generation</b> | ReportLab generates structured clinical reports with embedded mask overlay and QR code.      |
| <b>QR Code Sharing</b>       | Scannable QR code to access the system in mobile                                             |
| <b>History Management</b>    | localStorage-based analysis log enables review of previous scans without a server database.  |
| <b>Dashboard Analytics</b>   | Recharts visualisation of prediction trends and class distributions from session history.    |
| <b>Tumor Area Estimation</b> | U-Net mask post-processed to compute tumor area as a percentage of total scan area.          |
| <b>CPU-Only Deployment</b>   | All three models trained and served on CPU; no GPU required for inference.                   |
| <b>Modern Responsive UI</b>  | Tailwind CSS + Framer Motion deliver a polished, animated interface across all screen sizes. |

Table 13.1 – System feature catalogue

## 14 • Challenges Faced

| Challenge                     | Solution Applied                                                                                                                     |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| BraTS Dataset (~13 GB)        | Used another dataset to fit within CPU/RAM constraints while maintaining coverage of tumor phenotypes.                               |
| Class Imbalance (Meningioma)  | Applied class-weighted loss functions and targeted augmentation for minority class samples.                                          |
| Overfitting on Small Datasets | Dropout (0.5 for CNN, 0.3 for MobileNetV2), Batch Normalisation (U-Net), EarlyStopping (patience=5), ReduceLROnPlateau (factor=0.5). |
| Library Dependency Conflicts  | Isolated virtual environments with pinned dependency versions (requirements.txt) per development phase.                              |
| CPU-Only Training             | Batch sizes tuned per model (433 / 175 / 307). Transfer learning for MobileNetV2 drastically cut required training from scratch.     |

Table 14.1 – Challenges and solutions



## 15 • Future Scope

### Model Enhancements

- Grad-CAM Explainability — Visual saliency maps showing which MRI regions influenced each prediction; essential for clinical trust and regulatory pathways.
- DICOM Format Support — Native .dcm file input for direct integration with hospital PACS systems.
- Vision Transformer Models — Swin Transformer or ViT architectures as replacements or ensembles with current CNN models.
- Model Optimisation — Pruning, quantisation, and knowledge distillation for edge device deployment.

### Application & Deployment

- Cloud Deployment — AWS / GCP / Azure with auto-scaling, HTTPS, and CDN for global production access.
- GPU-Accelerated Inference — Sub-second latency enabling real-time clinical integration.
- Mobile Application — Native Android / iOS apps for on-the-go MRI analysis.
- User Authentication & RBAC — Secure login with role-based access control for multi-user clinical deployment.
- Database Integration — PostgreSQL backend replacing localStorage for persistent multi-session history and audit trails.
- Enhanced Reports — Tumor size in mm, anatomical location, longitudinal growth rate, and AI-generated clinical summary text.

## 16 • Conclusion

This project successfully delivers a fully functional, end-to-end AI-powered brain tumor analysis system integrating three specialised deep learning models into a unified clinical diagnostic pipeline.

| # | Deliverable                   | Metric                          | Status      |
|---|-------------------------------|---------------------------------|-------------|
| 1 | CNN Binary Detector           | 97.3% test accuracy, F1 = 0.97  | Complete    |
| 2 | MobileNetV2 Classifier        | 90.0% accuracy, macro F1 = 0.90 | Complete    |
| 3 | U-Net Segmentation            | Dice = 0.8041 validation        | Complete    |
| 4 | FastAPI Backend (5 endpoints) | Full CRUD + PDF generation      | Operational |
| 5 | React Frontend                | Batch, history, QR, dashboard   | Operational |

Table 16.1 – Project deliverables summary

All seven project objectives have been met. The system demonstrates that production-grade AI medical imaging pipelines can be developed and deployed effectively within CPU-only training constraints, through careful architectural choices, hyperparameter tuning, and systematic regularisation. The project establishes a strong technical foundation for future clinical deployment with Grad-CAM explainability, DICOM support, and cloud scaling.

**Detection Accuracy:** 97.3% (CNN Binary Classifier, 1,924 test samples)

**Classification Accuracy:** 90.0% (MobileNetV2, 4 classes, macro F1 = 0.90)

**Segmentation Dice Score:** 0.80+ (U-Net, Val Dice = 0.8041, Train Dice = 0.9164)

**Training Dataset Size:** ~22,946 images across three datasets

**Deployment Stack:** React.js • FastAPI • TensorFlow 2.x • ReportLab